

LAPORAN UTS PRAKTIKUM PEMROGRAMAN BACKEND LANJUT

RESTful API SIAKAD Mini



Octavia Nuzulul Azmi

434241040

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS VOKASI

UNIVERSITAS AIRLANGGA

SURABAYA

2026

Contents

BAB I PENDAHULUAN	3
A. Latar Belakang	3
B. Tujuan	3
C. Ruang Lingkup	3
BAB II TEKNOLOGI DAN STRUKTUR PROJECT	4
A. Teknologi yang Digunakan	4
B. Struktur Project	4
C. Version Control	5
BAB III DATABASE DAN MODEL	6
A. Struktur Tabel	6
B. Relasi Antar Tabel	7
C. Migration dan Seeder	8
BAB IV AUTHENTICATION DAN AUTHORIZATION	9
A. Authentication	9
B. Authorization	9
C. Rate Limiting Login	9
BAB V IMPLEMENTASI DAN PENGUJIAN ENDPOINT	10
A. Authentication	10
B. Manajemen Mahasiswa	11
C. Mata Kuliah	15
D. KRS atau Enrollment	15
BAB VI BUSINESS RULE, VALIDATION, DAN ERROR HANDLING	18
A. Business Rule	18
B. Validation	18
C. Format Response	18
D. Status Code	19
BAB VII PENGUJIAN	20
A. Metode Pengujian	20
B. Hasil Pengujian	20
C. Pembahasan Hasil Pengujian	20
BAB VIII KESIMPULAN	21
DAFTAR PUSTAKA	22

Repository GitHub: <https://github.com/octaviaazmi/tugas-backend-fiber/tree/main/UTS>

BAB I

PENDAHULUAN

A. Latar Belakang

SIAKAD Mini adalah layanan akademik sederhana yang mengelola data mahasiswa, mata kuliah, dan Kartu Rencana Studi (KRS). API ini digunakan oleh dua peran, yaitu admin dan mahasiswa. Admin dapat mengelola data mahasiswa dan mata kuliah, sedangkan mahasiswa dapat melihat profil sendiri serta mengambil atau membatalkan mata kuliah pada KRS miliknya.

Project ini merupakan tugas UTS mata kuliah Pemrograman Backend Lanjut dengan 10 endpoint yang sudah ditentukan pada studi kasus.

B. Tujuan

1. Membangun RESTful API SIAKAD Mini menggunakan Go, Fiber, dan PostgreSQL.
2. Menerapkan authentication menggunakan token dan authorization berdasarkan role.
3. Menerapkan business rule akademik pada proses pengambilan KRS.
4. Menyediakan format response yang seragam.

C. Ruang Lingkup

API terdiri dari 10 endpoint yang dibangun menggunakan Go dan framework Fiber serta menggunakan PostgreSQL sebagai database. Seeder mencakup 1 admin, 20 mahasiswa, dan 10 mata kuliah. Pengujian endpoint dilakukan menggunakan Postman.

BAB II

TEKNOLOGI DAN STRUKTUR PROJECT

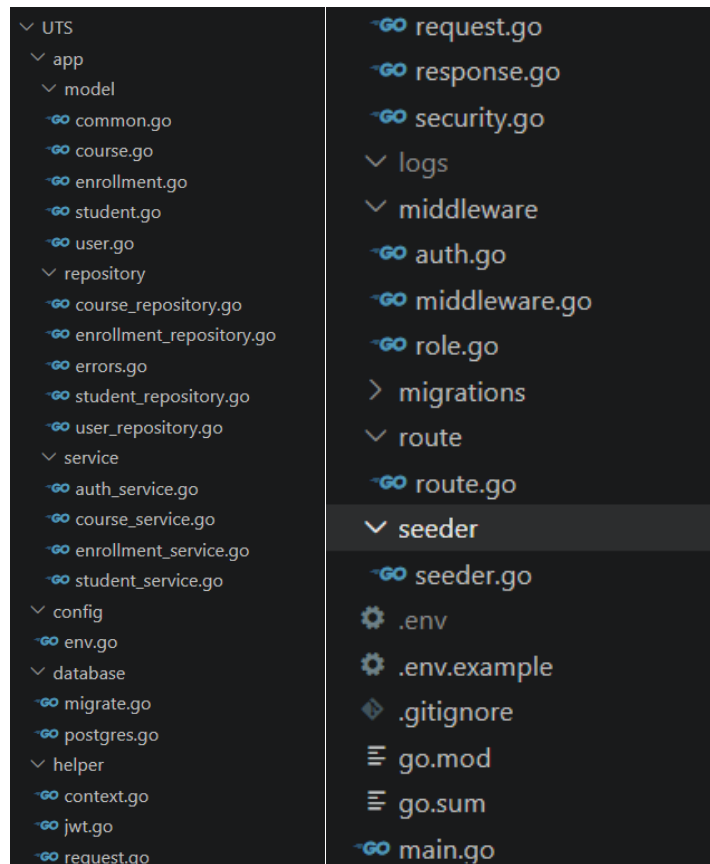
A. Teknologi yang Digunakan

1. Go: Go digunakan sebagai bahasa pemrograman utama untuk membangun RESTful API pada project ini.
2. Fiber: Fiber digunakan sebagai framework web untuk menangani routing, middleware, dan parsing request.
3. pgx: pgx digunakan sebagai driver PostgreSQL untuk berinteraksi dengan database menggunakan query SQL yang dikirim sebagai parameter.
4. PostgreSQL: PostgreSQL digunakan sebagai database untuk menyimpan data user, mahasiswa, mata kuliah, dan enrollment.
5. JWT: JWT digunakan sebagai authentication token setelah user berhasil melakukan login.
6. bcrypt: bcrypt digunakan untuk melakukan hashing terhadap password sebelum disimpan ke database.
7. godotenv: godotenv digunakan untuk membaca konfigurasi aplikasi dari file .env.
8. Postman: Postman digunakan untuk melakukan pengujian terhadap endpoint yang tersedia.
9. Git dan GitHub: Git dan GitHub digunakan sebagai version control untuk menyimpan dan mengelola perubahan pada project.

B. Struktur Project

Project disusun ke dalam beberapa folder berdasarkan fungsi masing-masing.

1. app/model/: Folder ini berisi struct model dan struct request/response.
2. app/repository/: Folder ini berisi kode akses data, yaitu query SQL ke database melalui pgx.
3. app/service/: Folder ini berisi business rules dan handler untuk setiap endpoint.
4. config/: Folder ini digunakan untuk membaca konfigurasi dari file .env.
5. database/: Folder ini menangani koneksi database dan menjalankan migrasi.
6. helper/: Folder ini berisi fungsi bantuan seperti JWT, hashing password, dan format response.
7. middleware/: Folder ini berisi middleware untuk authentication, authorization, dan logging.
8. route/: Folder ini berisi pendaftaran route untuk setiap endpoint.
9. seeder/: Folder ini digunakan untuk membuat data awal pada database.
10. migrations/: Folder ini berisi file SQL untuk membuat tabel.
11. main.go: File ini merupakan entry point aplikasi.



Gambar 2.1 Struktur Folder

C. Version Control

Project dikerjakan menggunakan Git dan disimpan pada repository GitHub secara bertahap. Perubahan project dilakukan melalui beberapa commit, mulai dari setup project, migration dan seeder, authentication, pengelolaan mahasiswa, mata kuliah, hingga enrollment.

Commits on Oct 4, 2026

UTS: route, main, dan seeder (admin, 20 mhs, 10 MK) octaviaazmi committed 2 minutes ago	85a0ee0		
UTS: service layer auth, student, course, dan enrollment octaviaazmi committed 53 minutes ago	81509e7		
UTS: middleware auth, role, logger, CORS, dan RequireJSON octaviaazmi committed 1 hour ago	e12252a		
UTS: repository layer untuk user, student, course, enrollment octaviaazmi committed 1 hour ago	04d5b98		
UTS: helper response, security, jwt, context, dan query parser octaviaazmi committed 1 hour ago	8152503		
UTS: migration SQL dan model structs octaviaazmi committed 1 hour ago	8fcb581		
UTS: setup project Fiber + pgx + config octaviaazmi committed 1 hour ago	8e9d767		

Gambar 2.2 Riwayat Commit di Github

BAB III

DATABASE DAN MODEL

A. Struktur Tabel

1. Tabel users

Tabel users digunakan untuk menyimpan akun yang dapat melakukan login. Tabel ini memiliki kolom id, email, password, role, created_at, dan updated_at. Email dibuat unik, sedangkan password disimpan dalam bentuk hash bcrypt. Role terdiri dari admin dan mahasiswa.

```
siakad_mini=# \d users
```

Column	Type	Collation	Nullable	Default
id	bigint		not null	nextval('users_id_seq'::regclass)
email	character varying(100)		not null	
password	character varying(255)		not null	
role	character varying(20)		not null	
created_at	timestamp with time zone			
updated_at	timestamp with time zone			

Indexes:

- "users_pkey" PRIMARY KEY, btree (id)
- "idx_users_email" UNIQUE, btree (email)

Check constraints:

- "chk_users_role" CHECK (role::text = ANY (ARRAY['admin'::character varying, 'mahasiswa'::character varying]::text[]))

Referenced by:

- TABLE "students" CONSTRAINT "fk_users_student" FOREIGN KEY (user_id) REFERENCES users(id)

Gambar 3.1 Struktur tabel users

2. Tabel students

Tabel students digunakan untuk menyimpan data akademik mahasiswa. Tabel ini memiliki kolom id, user_id, nim, nama, prodi, angkatan, ipk_terakhir, deleted_at, dan timestamp. Kolom user_id memiliki relasi dengan tabel users, sedangkan nim dibuat unik.

```
siakad_mini=# \d students
```

Column	Type	Collation	Nullable	Default
id	bigint		not null	nextval('students_id_seq'::regclass)
user_id	bigint		not null	
nim	character varying(12)		not null	
nama	character varying(100)		not null	
prodi	character varying(100)		not null	
angkatan	bigint		not null	
ipk_terakhir	numeric(3,2)			0
deleted_at	timestamp with time zone			
created_at	timestamp with time zone			
updated_at	timestamp with time zone			

Indexes:

- "students_pkey" PRIMARY KEY, btree (id)
- "idx_students_deleted_at" btree (deleted_at)
- "idx_students_nim" UNIQUE, btree (nim)
- "idx_students_user_id" UNIQUE, btree (user_id)

Foreign-key constraints:

- "fk_users_student" FOREIGN KEY (user_id) REFERENCES users(id)

Referenced by:

- TABLE "enrollments" CONSTRAINT "fk_students_enrollments" FOREIGN KEY (student_id) REFERENCES students(id)

Gambar 3.2 Struktur tabel students

3. Tabel courses

Tabel courses digunakan untuk menyimpan data mata kuliah. Tabel ini memiliki kolom id, kode_mk, nama_mk, sks, semester, dan kuota. Kode mata kuliah dibuat unik.

```
siakad_mini=# \d courses
```

Column	Type	Collation	Nullable	Default
id	bigint		not null	nextval('courses_id_seq'::regclass)
kode_mk	character varying(20)		not null	
nama_mk	character varying(150)		not null	
sks	bigint		not null	
semester	bigint		not null	
kuota	bigint		not null	
created_at	timestamp with time zone			
updated_at	timestamp with time zone			

Indexes:
 "courses_pkey" PRIMARY KEY, btree (id)
 "idx_courses_kode_mk" UNIQUE, btree (kode_mk)

Referenced by:
 TABLE "enrollments" CONSTRAINT "fk_courses_enrollments" FOREIGN KEY (course_id) REFERENCES courses(id)

Gambar 3.3 Struktur tabel courses

4. Tabel enrollments

Tabel enrollments digunakan untuk menyimpan data pengambilan mata kuliah oleh mahasiswa. Tabel ini memiliki kolom id, student_id, course_id, tahun_akademik, dan created_at. Kombinasi student_id, course_id, dan tahun_akademik dibuat unik untuk mencegah mahasiswa mengambil mata kuliah yang sama pada tahun akademik yang sama.

```
siakad_mini=# \d enrollments
```

Column	Type	Collation	Nullable	Default
id	bigint		not null	nextval('enrollments_id_seq'::regclass)
student_id	bigint		not null	
course_id	bigint		not null	
tahun_akademik	character varying(20)		not null	
created_at	timestamp with time zone			

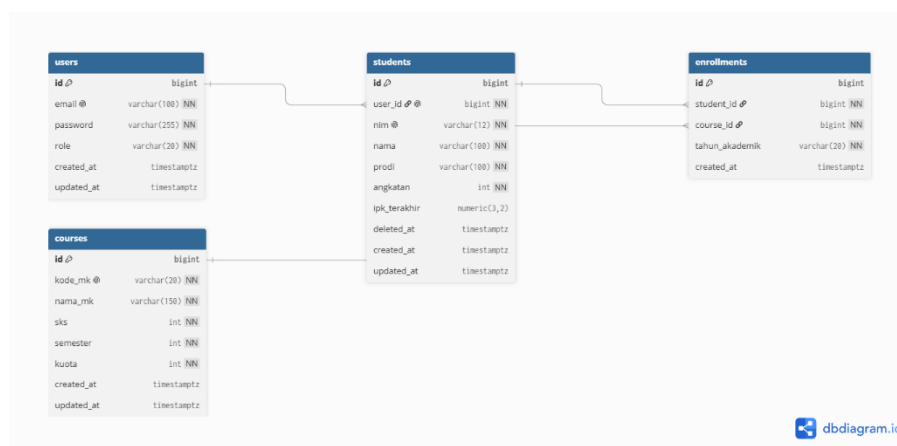
Indexes:
 "enrollments_pkey" PRIMARY KEY, btree (id)
 "idx_student_course_year" UNIQUE, btree (student_id, course_id, tahun_akademik)

Foreign-key constraints:
 "fk_courses_enrollments" FOREIGN KEY (course_id) REFERENCES courses(id)
 "fk_students_enrollments" FOREIGN KEY (student_id) REFERENCES students(id)

Gambar 3.4 Struktur tabel enrollments

B. Relasi Antar Tabel

Database memiliki beberapa relasi antar tabel. Tabel users berelasi dengan students, sedangkan students dan courses berelasi melalui tabel enrollments. Relasi tersebut memungkinkan satu mahasiswa memiliki beberapa enrollment dan satu mata kuliah diambil oleh beberapa mahasiswa.



Gambar 3.5 relasi tabel

C. Migration dan Seeder

Migration dijalankan menggunakan file SQL yang tersimpan di folder migrations. File-file SQL ini dijalankan secara otomatis ketika aplikasi pertama kali dijalankan, dan hanya membuat tabel jika tabel tersebut belum ada. Seeder dijalankan setelah migration dan hanya membuat data jika data tersebut belum tersedia. Hal ini mencegah data yang sama dibuat berulang kali ketika aplikasi dijalankan kembali.

Data awal yang dibuat terdiri dari 1 akun admin, 20 mahasiswa, dan 10 mata kuliah. Akun admin menggunakan email `admin@siakad.test`, sedangkan password awal mahasiswa menggunakan NIM masing-masing.

```
{
  "time": "2026-10-04T03:24:01.0155174+07:00",
  "level": "INFO",
  "msg": "http_request",
  "request_id": "18f98b43-dd32-48fb-b8c8-84c346c786c1",
  "method": "GET",
  "path": "/api/v1/health",
  "status": 200,
  "duration": 767900,
  "ip": "127.0.0.1"
}
{
  "time": "2026-10-04T03:25:15.2050048+07:00",
  "level": "INFO",
  "msg": "http_request",
  "request_id": "19f98b43-dd32-48fb-b8c8-84c346c786c1",
  "method": "POST",
  "path": "/api/v1/auth/login",
  "status": 200,
  "duration": 312029000,
  "ip": "127.0.0.1"
}
{
  "time": "2026-10-04T03:26:15.0719889+07:00",
  "level": "INFO",
  "msg": "http_request",
  "request_id": "1af98b43-dd32-48fb-b8c8-84c346c786c1",
  "method": "GET",
  "path": "/api/v1/students",
  "status": 200,
  "duration": 7034300,
  "ip": "127.0.0.1",
  "user_id": 1,
  "role": "admin"
}
{
  "time": "2026-10-04T04:10:46.4908698+07:00",
  "level": "INFO",
  "msg": "sinyal berhenti diterima, menutup server"
}
{
  "time": "2026-10-04T04:10:46.4947788+07:00",
  "level": "INFO",
  "msg": "server berhenti dengan rapi"
}
```

Gambar 3.6 Log migration dan seeder

BAB IV

AUTHENTICATION DAN AUTHORIZATION

A. Authentication

Sistem menggunakan JWT sebagai authentication token. Setelah user berhasil login, server mengembalikan access token yang digunakan untuk mengakses endpoint yang membutuhkan authentication. Token dikirim melalui header Authorization dengan format Bearer token. Token berisi informasi user_id, email, role, dan waktu kadaluarsa. Masa berlaku token secara default adalah 60 menit. Password user tidak disimpan dalam bentuk teks biasa. Password disimpan dalam bentuk hash menggunakan bcrypt. Saat proses login, password yang dikirim akan dibandingkan dengan hash yang tersimpan di database.

B. Authorization

Authorization digunakan untuk membatasi akses berdasarkan role user. Endpoint login dapat diakses tanpa token. Endpoint tertentu dapat diakses oleh semua user yang sudah login, sedangkan endpoint pengelolaan mahasiswa hanya dapat diakses oleh admin dan endpoint enrollment hanya dapat diakses oleh mahasiswa. Jika role user tidak sesuai dengan akses endpoint, sistem mengembalikan response 403 Forbidden.

C. Rate Limiting Login

Endpoint login memiliki batas maksimal 5 kali percobaan gagal dalam satu menit berdasarkan IP. Jika batas tersebut terlampaui, sistem mengembalikan response 429 Too Many Requests.

BAB V

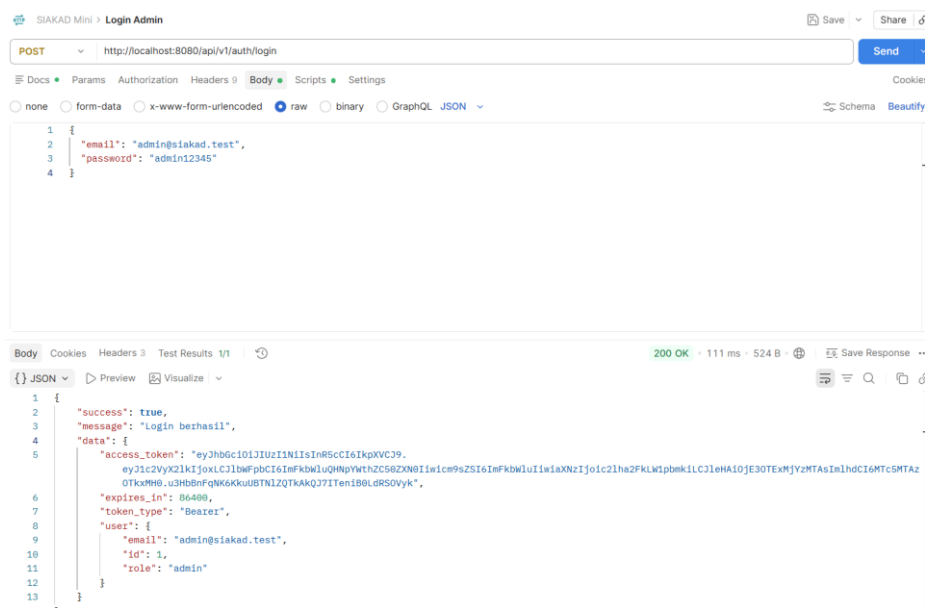
IMPLEMENTASI DAN PENGUJIAN ENDPOINT

A. Authentication

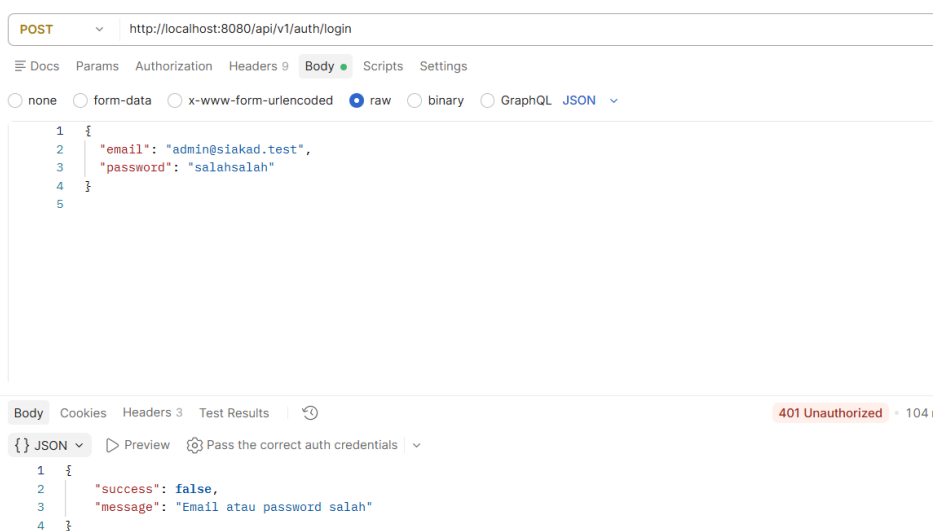
1. POST /api/v1/auth/login

Endpoint ini digunakan untuk melakukan login. User mengirimkan email dan password, kemudian sistem memeriksa data tersebut. Jika credential benar, sistem mengembalikan access token yang dapat digunakan untuk mengakses endpoint yang membutuhkan authentication.

Pengujian dilakukan menggunakan credential yang valid untuk memastikan proses login berhasil. Pengujian dengan credential yang salah juga dilakukan untuk memastikan sistem menolak login yang tidak valid.



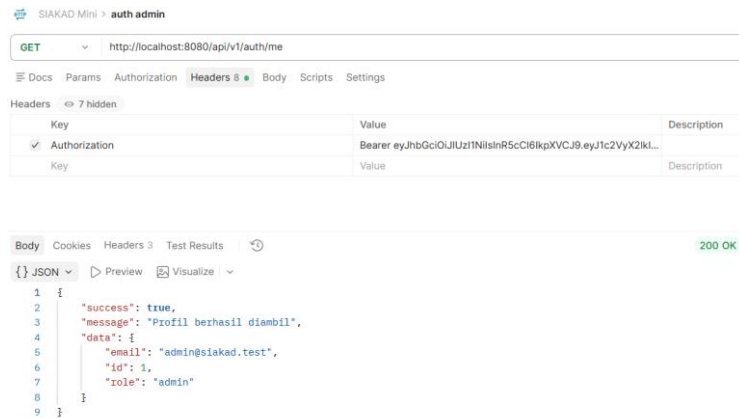
Gambar 5.1 Pengujian POST /api/v1/auth/login berhasil



Gambar 5.2 Pengujian POST /api/v1/auth/login gagal

2. GET /api/v1/auth/me

Endpoint ini digunakan untuk mengambil profil user yang sedang login. Jika user memiliki role mahasiswa, response juga menampilkan data mahasiswa yang berkaitan dengan akun tersebut. Pengujian dilakukan untuk memastikan endpoint dapat mengambil profil user berdasarkan token yang digunakan.

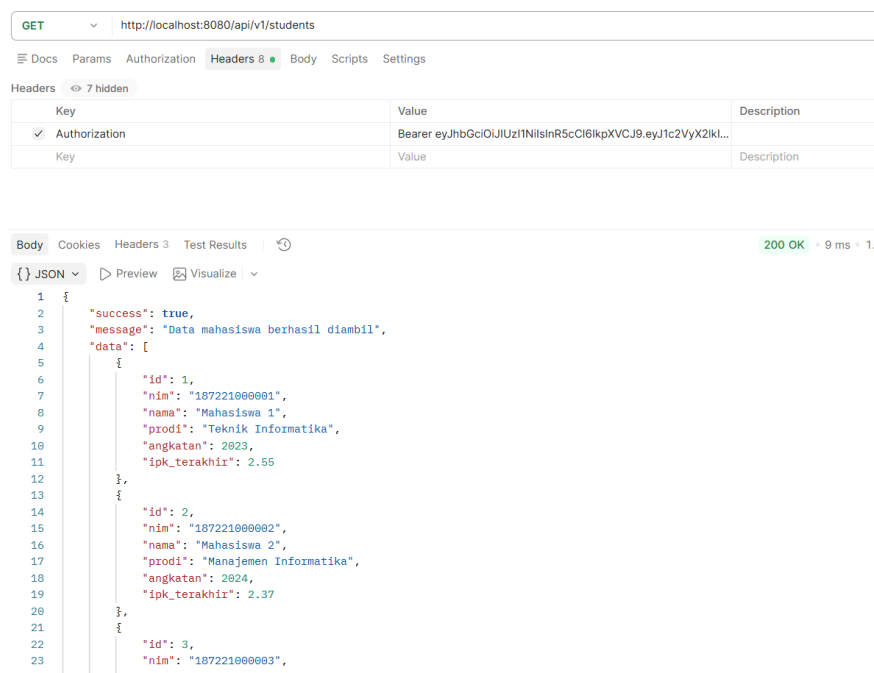


Gambar 5.3 Pengujian GET /api/v1/auth/me

B. Manajemen Mahasiswa

1. GET /api/v1/students

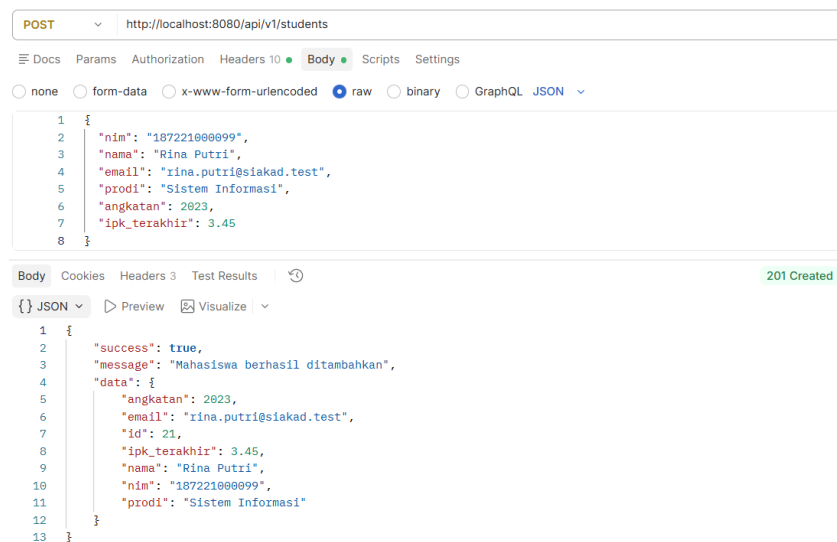
Endpoint ini digunakan untuk mengambil daftar mahasiswa. Endpoint mendukung pagination, filtering berdasarkan prodi dan angkatan, search berdasarkan NIM atau nama, serta sorting berdasarkan nama atau IPK. Pengujian dilakukan untuk memastikan daftar mahasiswa dapat ditampilkan dan parameter yang tersedia dapat digunakan sesuai kebutuhan.



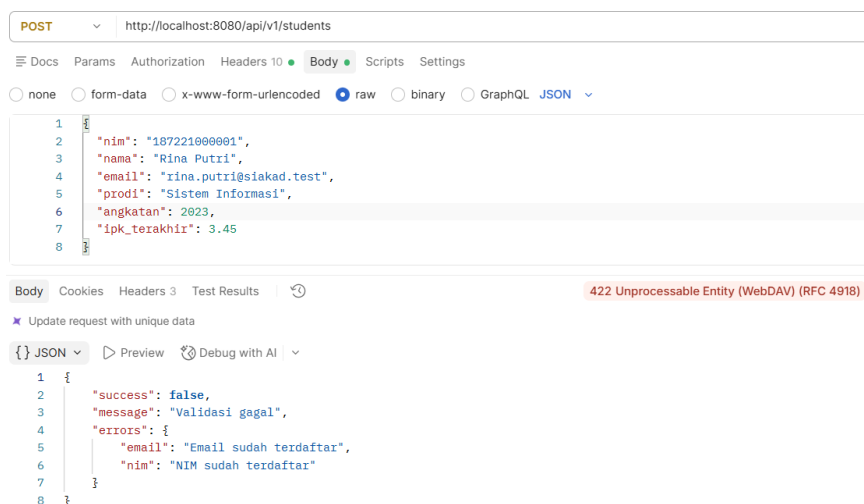
Gambar 5.4 Pengujian GET /api/v1/students

2. POST /api/v1/students

Endpoint ini digunakan untuk menambahkan mahasiswa baru sekaligus membuat akun user untuk mahasiswa tersebut. Proses dilakukan dalam satu transaction. Sistem membuat user terlebih dahulu, kemudian membuat data mahasiswa yang terhubung. Jika salah satu langkah gagal, seluruh perubahan dibatalkan sehingga tidak ada data yang tersimpan sebagian. Pengujian dilakukan menggunakan data mahasiswa yang valid dan data yang tidak valid, termasuk data dengan NIM atau email yang sudah digunakan.



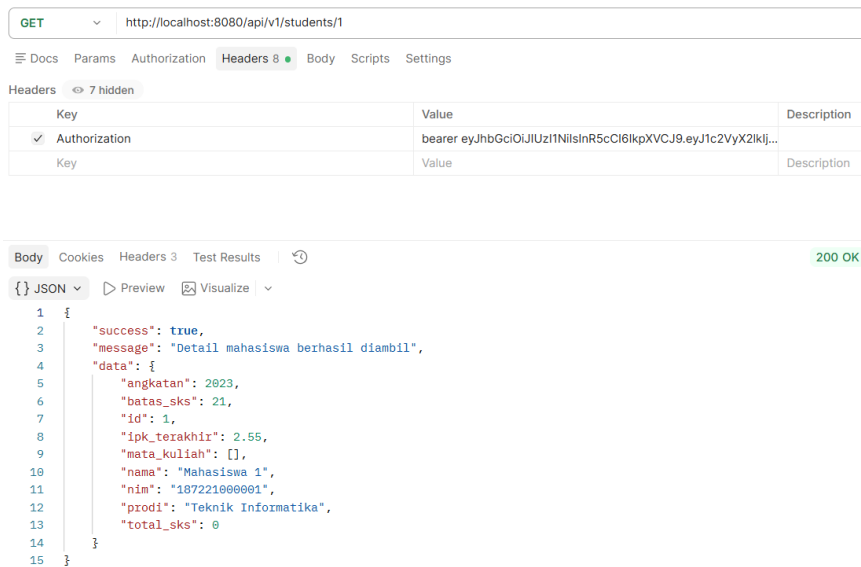
Gambar 5.5 Pengujian POST /api/v1/students berhasil



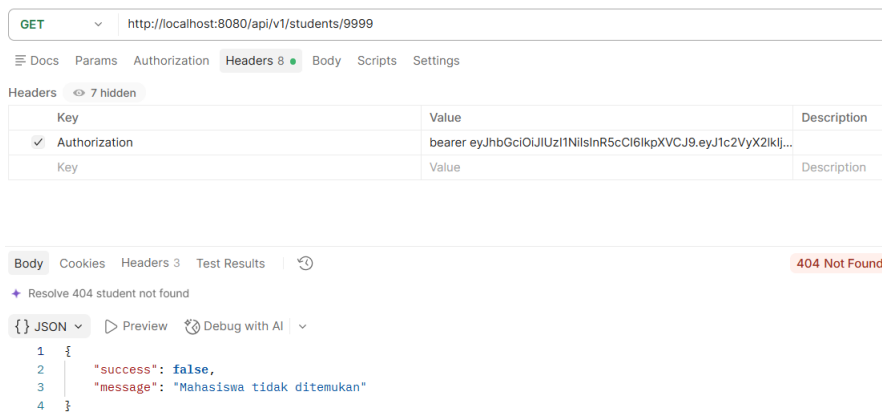
Gambar 5.6 Pengujian POST /api/v1/students gagal

3. GET /api/v1/students/{id}

Endpoint ini digunakan untuk mengambil detail mahasiswa beserta informasi mata kuliah yang diambil, total SKS, dan batas SKS. Admin dapat melihat data mahasiswa, sedangkan mahasiswa hanya dapat melihat data miliknya sendiri. Pengujian dilakukan untuk kondisi akses yang diperbolehkan dan kondisi ketika mahasiswa mencoba mengakses data mahasiswa lain.



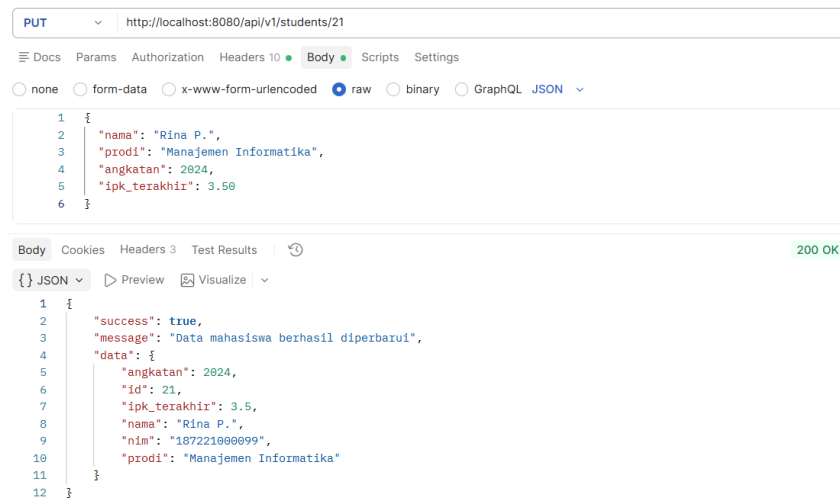
Gambar 5.7 Pengujian GET /api/v1/students/{id} berhasil



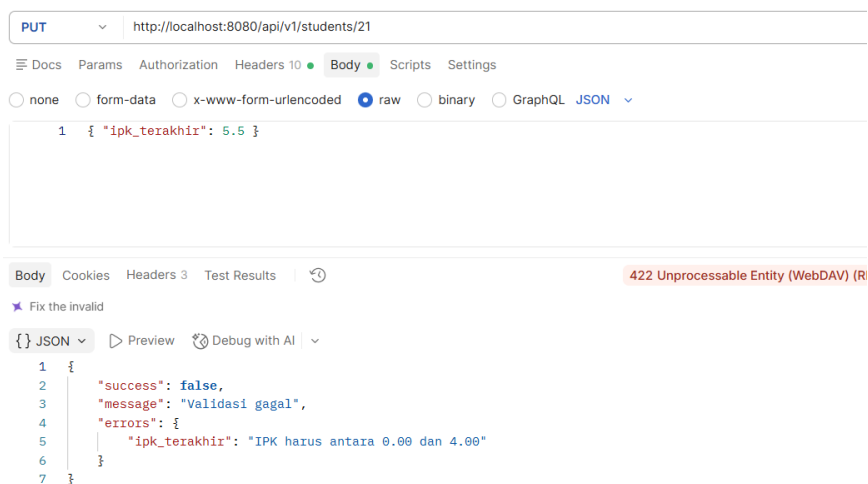
Gambar 5.8 Pengujian akses mahasiswa ke data mahasiswa lain

4. PUT /api/v1/students/{id}

Endpoint ini digunakan untuk memperbarui data mahasiswa. Data yang dapat diperbarui meliputi nama, prodi, angkatan, dan IPK. NIM tidak dapat diubah. Pengujian dilakukan menggunakan data yang valid dan data yang tidak sesuai dengan validation.



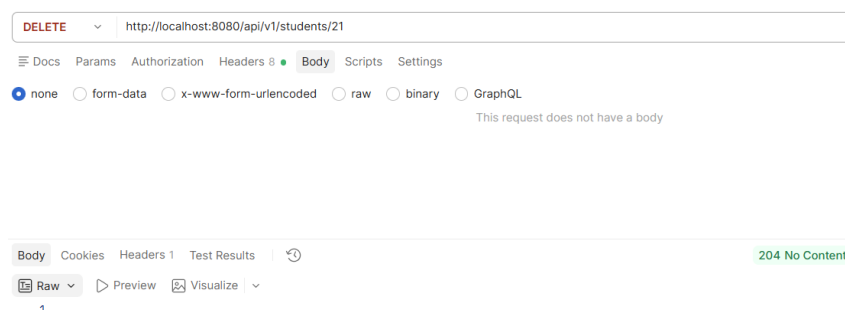
Gambar 5.9 Pengujian PUT /api/v1/students/{id}



Gambar 5.10 Pengujian negatif IPK > 4

5. DELETE /api/v1/students/{id}

Endpoint ini digunakan untuk melakukan soft delete terhadap data mahasiswa. Data tidak langsung dihapus dari database, tetapi kolom deleted_at diisi. Mahasiswa yang sudah di-soft delete tidak muncul dalam daftar mahasiswa dan tidak dapat melakukan login.

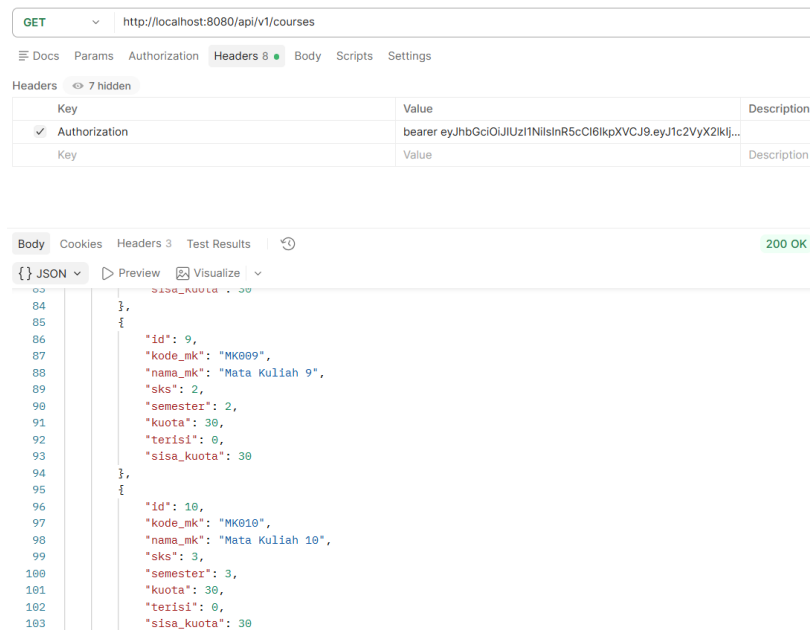


Gambar 5.11 Pengujian DELETE /api/v1/students/{id}

C. Mata Kuliah

1. GET /api/v1/courses

Endpoint ini digunakan untuk mengambil daftar mata kuliah. Endpoint mendukung filtering berdasarkan semester, search berdasarkan kode atau nama mata kuliah, serta filtering mata kuliah yang masih memiliki kuota. Response juga menampilkan jumlah mahasiswa yang sudah mengambil mata kuliah dan sisa kuota yang tersedia.



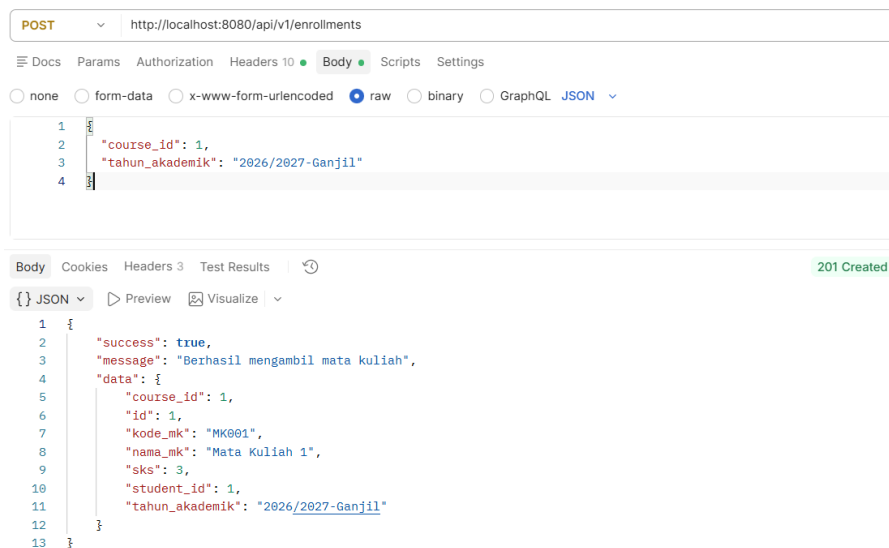
Gambar 5.12 Pengujian GET /api/v1/courses

D. KRS atau Enrollment

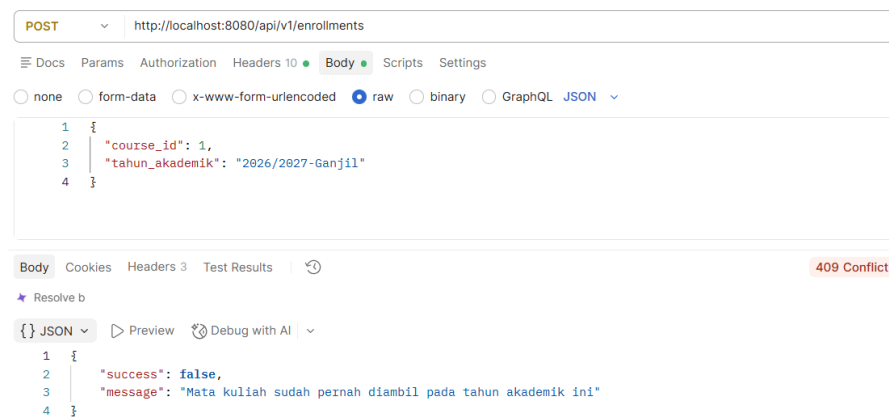
1. POST /api/v1/enrollments

Endpoint ini digunakan oleh mahasiswa untuk mengambil mata kuliah. Sebelum enrollment dibuat, sistem melakukan beberapa pengecekan, yaitu memastikan mata kuliah belum pernah diambil pada tahun akademik yang sama, memeriksa kuota, dan memeriksa batas SKS mahasiswa.

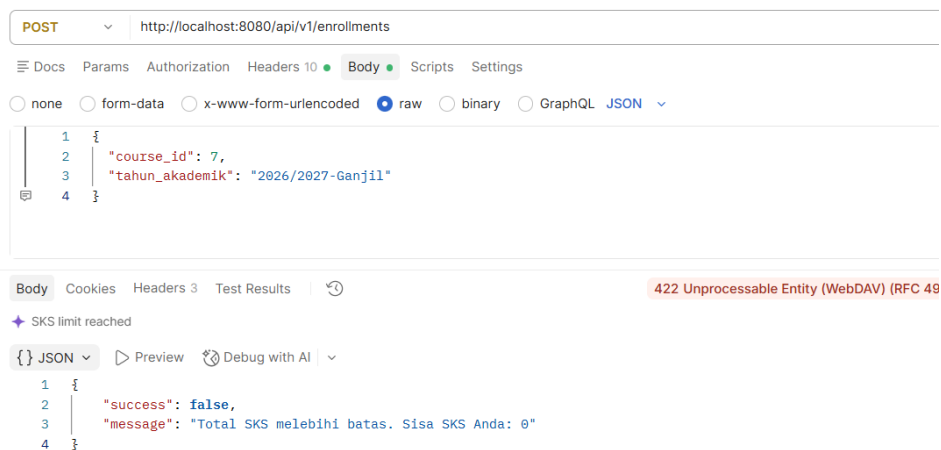
Proses enrollment dilakukan dalam satu transaction. Row locking digunakan untuk mencegah masalah ketika beberapa mahasiswa mencoba mengambil mata kuliah yang sama pada waktu yang bersamaan. Pengujian dilakukan untuk kondisi berhasil dan beberapa kondisi yang melanggar business rule.



Gambar 5.13 Pengujian POST /api/v1/enrollments berhasil



Gambar 5.14 Pengujian POST /api/v1/enrollments duplikat

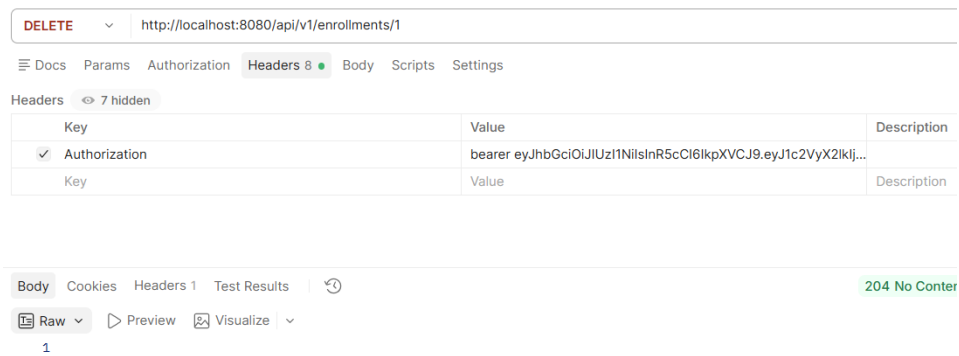


Gambar 5.15 Pengujian POST /api/v1/enrollments melebihi batas SKS

2. DELETE /api/v1/enrollments/{id}

Endpoint ini digunakan oleh mahasiswa untuk membatalkan mata kuliah yang sudah diambil pada KRS. Sistem memastikan enrollment tersebut merupakan milik mahasiswa yang sedang

login. Setelah enrollment berhasil dihapus, jumlah mahasiswa yang mengambil mata kuliah akan berkurang sehingga sisa kuota kembali bertambah.



Gambar 5.16 Pengujian DELETE `/api/v1/enrollments/{id}`

BAB VI

BUSINESS RULE, VALIDATION, DAN ERROR HANDLING

A. Business Rule

1. Batas SKS berdasarkan IPK

Batas SKS mahasiswa ditentukan berdasarkan IPK terakhir.

- a. IPK 3.00 atau lebih memiliki batas maksimal 24 SKS.
- b. IPK 2.50 sampai 2.99 memiliki batas maksimal 21 SKS.
- c. IPK di bawah 2.50 memiliki batas maksimal 18 SKS.

Sistem menghitung total SKS yang sudah diambil mahasiswa pada tahun akademik tersebut sebelum menentukan apakah mata kuliah baru dapat diambil.

2. Larangan Duplikasi Mata Kuliah

Mahasiswa tidak dapat mengambil mata kuliah yang sama pada tahun akademik yang sama. Pengecekan dilakukan pada service layer dan juga diperkuat dengan unique constraint pada database.

3. Batas Kuota Mata Kuliah

Setiap mata kuliah memiliki jumlah kuota tertentu. Jika kuota sudah penuh, mahasiswa tidak dapat mengambil mata kuliah tersebut.

Row locking digunakan pada proses enrollment untuk mengurangi risiko race condition ketika terdapat beberapa request yang masuk secara bersamaan.

4. Kepemilikan KRS

Mahasiswa hanya dapat mengakses dan membatalkan enrollment miliknya sendiri. Sistem melakukan pengecekan terhadap `student_id` sebelum memberikan akses.

B. Validation

Validation dilakukan untuk memastikan data yang dikirim melalui request sesuai dengan aturan yang ditentukan. Beberapa validation yang diterapkan meliputi format email, panjang password, NIM 12 digit, format tahun angkatan, nilai IPK antara 0.00 sampai 4.00, serta format tahun akademik. Jika validation gagal, sistem mengembalikan response error dengan status 422.

C. Format Response

Response pada API dibuat dengan format yang seragam. Response sukses memiliki informasi success, message, dan data. Endpoint yang menggunakan pagination juga memiliki informasi meta. Response error memiliki informasi success, message, dan errors jika terdapat kesalahan pada field tertentu.

D. Status Code

Status code yang digunakan dalam project meliputi 200, 201, 204, 401, 403, 404, 409, 422, 429, dan 500. Penggunaan status code disesuaikan dengan kondisi yang terjadi pada request.

BAB VII

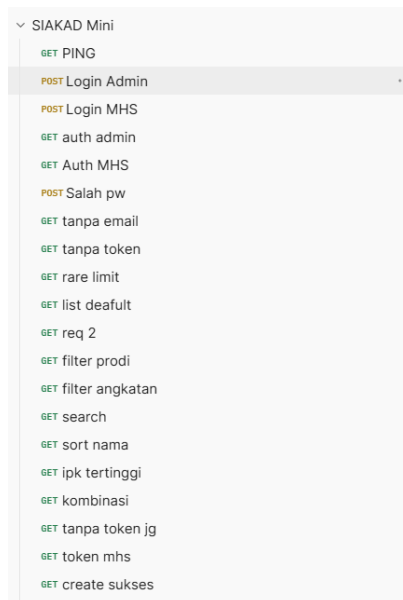
PENGUJIAN

A. Metode Pengujian

Pengujian dilakukan secara manual menggunakan Postman. Pengujian mencakup kondisi berhasil dan kondisi error untuk memastikan endpoint dapat menangani request sesuai dengan aturan yang diterapkan. Pengujian tidak menyalin seluruh request body dan response ke dalam laporan. Request dan response ditampilkan melalui screenshot Postman pada pembahasan masing-masing endpoint.

B. Hasil Pengujian

Berdasarkan pengujian yang dilakukan, endpoint dapat digunakan sesuai dengan fungsi masing-masing. Authentication dan authorization dapat membatasi akses berdasarkan token dan role. Business rule seperti batas SKS, larangan duplikasi, batas kuota, dan kepemilikan KRS juga diuji melalui beberapa kondisi.



Gambar 7.1 Collection pengujian Postman

C. Pembahasan Hasil Pengujian

Pengujian menunjukkan bahwa proses authentication dapat membedakan credential yang valid dan tidak valid. Authorization juga dapat membatasi akses endpoint berdasarkan role user. Pada proses pengelolaan mahasiswa, sistem dapat menambahkan, menampilkan, memperbarui, dan melakukan soft delete terhadap data mahasiswa sesuai dengan akses yang diberikan.

Pada proses enrollment, sistem dapat melakukan pengecekan terhadap duplikasi mata kuliah, kuota, batas SKS, dan kepemilikan data. Hal tersebut membantu memastikan proses pengambilan mata kuliah berjalan sesuai business rule yang diterapkan.

BAB VIII

KESIMPULAN

RESTful API SIAKAD Mini telah dibangun menggunakan Go, Fiber, dan PostgreSQL. API memiliki 10 endpoint yang mencakup authentication, pengelolaan mahasiswa, mata kuliah, dan enrollment. Authentication menggunakan JWT dan authorization berdasarkan role telah diterapkan untuk membatasi akses user. Business rule seperti batas SKS berdasarkan IPK, batas kuota, larangan duplikasi mata kuliah, dan kepemilikan KRS juga telah diterapkan.

Pengujian dilakukan menggunakan Postman dengan kondisi berhasil dan kondisi error. Hasil pengujian digunakan untuk memastikan setiap endpoint dapat menjalankan fungsi sesuai dengan kebutuhan project.

DAFTAR PUSTAKA

Fiber. (n.d.). *Fiber documentation*. <https://docs.gofiber.io/>

Go Authors. (n.d.). *The Go programming language documentation*. <https://go.dev/doc/>

JSON Web Tokens. (n.d.). *Introduction to JSON Web Tokens*. <https://jwt.io/introduction>

pgx. (n.d.). *pgx: PostgreSQL driver and toolkit for Go* [Computer software]. GitHub. <https://github.com/jackc/pgx>

PostgreSQL Global Development Group. (n.d.). *PostgreSQL 15 documentation*.
<https://www.postgresql.org/docs/15/>

The Go Authors. (n.d.). *bcrypt package* - golang.org/x/crypto/bcrypt. Go Packages.
<https://pkg.go.dev/golang.org/x/crypto/bcrypt>